FLOWJEEP — DATABASE ARCHITECTURE

System Specification, Schema Definition & Technical Handoff Document

Target Environment: Google Firebase Realtime Database (flowjeep-bf08a) | **Region:** asia-southeast1 (Singapore)
Local Timezone: Asia/Manila (UTC+8) | **Platform Version:** 2.1.0 Production

1. Executive Summary & Diagnostic Inspection

This document provides the canonical, ground-truth reference for the FlowJeep database architecture. The entire platform—comprising the React TypeScript web application, the Python OpenCV/YOLOv8 edge camera vision pipeline, and ESP32 paratransit hardware—relies exclusively on **Firebase Realtime Database (RTDB)**. Cloud Firestore is **NOT initialized or used** anywhere in the active codebase.

Database Configuration Details:

- **Instance URL:** https://flowjeep-bf08a-default-rtdb.asia-southeast1.firebasedatabase.app
- **Project ID:** flowjeep-bf08a
- **Authentication:** Firebase Authentication (email/password & Google OAuth) with profiles synced to /users/{uid}
- **Legacy Deprecation:** The legacy prototype node /vehicles has been completely removed. Live streaming telemetry is consolidated into /live_telemetry, while master vehicle attributes are stored in /jeepneys.

2. Canonical Database Structure (Node-by-Node Specification)

Path: /system_config

Purpose: Holds system-wide constants, paratransit fare matrix formulas, and time-to-live thresholds.

Fields: version (string), activeFareMatrix { baseFareFirst4km, succeedingKmRate, discountRate }, pingExpirationMinutes (number), gpsStaleThresholdSeconds (number)

Path: /users/{uid}

Purpose: Authenticated user profiles, roles, and operator cooperative assignments.

Fields: uid (string), email (string), name (string), role ('commuter'|'operator'|'admin'), cooperative (string?), operatorId (string?), assignedRouteId (string?), assignedPlate (string?), createdAt (number), lastLoginAt (number?)

Path: /routes/{routeId}

Purpose: Official LTFRB transit corridor registry, origin/destinations, base fares, and ordered stop sequences.

Fields: id (string), routeCode (string), name (string), origin (string), destination (string), totalDistanceKm (number), estimatedDurationMin (number), baseFare (number), status ('ACTIVE'|'INACTIVE'), color (string), stops (map of RouteStop)

Path: /jeepneys/{jeepneyId}

Purpose: Master vehicle registry for modern jeepneys. Stores permanent vehicle identity and hardware links.

Fields: id (string), plateNumber (string), cooperative (string), assignedRouteId (string), deviceId (string?), model (string), isAircon (boolean), totalSeats (number), standingAllowed (boolean), status ('IN_SERVICE'|'OFF_DUTY'|'MAINTENANCE')

Path: /devices/{deviceId}

Purpose: IoT hardware registry tracking edge units installed in vehicles.

Fields: deviceId (string), jeepneyId (string), esp32CamIp (string), firmwareVersion (string), cameraStatus ('STREAMING'|'DISCONNECTED'), dualBeamStatus ('CONNECTED'|'DISCONNECTED'), lastHeartbeat (number), plateNumber (string)

Path: /live_telemetry/{jeepneyId}

Purpose: High-frequency streaming telemetry: real-time GPS, speed, heading, and live passenger load.

Fields: jeepneyId (string), routeId (string), routeCode (string), routeName (string), plateNumber (string), currentCoord [lat, lng], headingDegrees (number), speedKmh (number), occupiedSeats (number), totalSeats (number), capacityPct (number), loadCategory (string), lastUpdate (number), isOnline (boolean)

Path: /doorway_events/{eventId}

Purpose: Chronological audit log of boarding/alighting events detected by YOLOv8n + ByteTrack.

Fields: eventId (string), jeepneyId (string), tripId (string?), eventType ('BOARDING'|'ALIGHTING'|'CANCELLED'), trackingId (number), confidence (number), currentOccupancy (number), timestamp (number), location [lat, lng]?

Path: /waiting_commuters/{pingId}

Purpose: Active transit wait pings submitted by commuters broadcasting stop locations.

Fields: pingId (string), userId (string), isGuest (boolean), routeId (string), pickupCoords { lat, lng }, destinationCoords { lat, lng }?, pickupStopName (string?), destinationName (string?), status ('waiting'|'acknowledged'|'expired'), timestamp (number), expiresAt (number)

Path: /trips/{tripId}

Purpose: Operating shift tracking for jeepney runs between terminals.

Fields: tripId (string), jeepneyId (string), routeId (string), driverUid (string?), startTime (number), endTime (number?), status ('IN_PROGRESS'|'COMPLETED'|'CANCELLED'), totalBoarded (number), totalAlighted (number), peakOccupancy (number)

Path: /demand_hotspots/{corridorId}/{id}

Purpose: Aggregated passenger concentration clusters along transit corridors for operator dispatch.

Fields: id (string), corridorId (string), stopName (string), lat (number), lng (number), waitingCount (number), demandLevel ('high'|'medium'|'low'), lastUpdated (number)

Path: /forecast_records/{corridorId}/{date}

Purpose: Persistent cache for pre-computed passenger load forecasts.

Fields: corridorId (string), dateString (string), weatherCondition (string), temperatureC (number), precipitationProb (number), peakPercentage (number), recommendedWindow (string), periods (map of PeriodRecord), lastModelRun (number)

3. Database Entity Implementation Matrix

Entity	Status	Location & Implementation Details
Users	IMPLEMENTED	RTDB /users/{uid}. Commuter and operator profiles.
Jeepneys	IMPLEMENTED	RTDB /jeepneys/{id}. Master registry (plate, capacity, model).
IoT Devices	IMPLEMENTED	RTDB /devices/{id}. DEV-ESP32-01 hardware unit.
Routes	IMPLEMENTED	RTDB /routes/{id}. Vector geometries in R-Tree.
Live Telemetry	IMPLEMENTED	RTDB /live_telemetry/{id}. Real-time GPS & passenger count.
Doorway Events	IMPLEMENTED	RTDB /doorway_events. YOLOv8n boarding/alighting logs.
Waiting Pings	IMPLEMENTED	RTDB /waiting_commuters. 20-min broadcasting pings.
Demand Hotspots	IMPLEMENTED	RTDB /demand_hotspots. Live heatmap circles.
Trips	PARTIAL	Service methods implemented in databaseService.ts.
Forecast Records	PARTIAL	Schema ready; Python Random Forest model runs in backend.
Weather	EXTERNAL API	NOT stored in DB. OpenWeather + Open-Meteo in-memory cache.
ETA	CALCULATED	NOT stored in DB. OSRM engine & paratransit dwell speeds.

4. Real-Time vs. Historical Data Classification

- **Real-Time Streaming:** /live_telemetry (GPS, speed, heading, passenger load) and /waiting_commuters (commuter broadcasts). Delivered via WebSocket sub-100ms listeners.
- **Historical & Audit:** /doorway_events (camera vision logs) and /trips (shift records). Preserved for auditing and ML retraining.
- **Master Reference:** /jeepneys, /devices, /routes, /system_config. Static metadata read on demand.
- **External / Not Stored:** Weather observations and forecasts are fetched dynamically and cached in client memory with a 10-minute TTL. They are never persisted to Firebase.

5. Weather Subsystem Architecture

FlowJeep employs a strict hybrid weather architecture that operates area-specifically without being constrained to transit corridors:
1. **Past Dates:** Open-Meteo Historical Weather API (archive-api.open-meteo.com) for observed actual conditions.
2. **Today (Current):** OpenWeather Current API (api.openweathermap.org/data/2.5/weather).
3. **Future Dates:** OpenWeather 5-Day / 3-Hour Forecast API (api.openweathermap.org/data/2.5/forecast).
Storage Decision: Weather data is deliberately **NOT stored in Firebase** to avoid bloat and redundant API cost. It is transformed into a unified format in hybridWeatherService.ts and cached in a lightweight in-memory JavaScript Map.

6. Passenger-Load Forecasting Subsystem

Passenger-load predictions combine temporal patterns with live weather impact factors:
- **Frontend Engine:** forecastService.ts resolves 6 baseline daily operating periods (Early Dispatch, Morning Rush, Midday Off-Peak, School Surge, Evening Homeward, Night Sweep) with weather-driven ridership advisories.
- **Python Backend Model:** iot_architecture/backend_server/rf_model_service.py hosts a Scikit-Learn RandomForestRegressor predicting load percentage from [day_of_week, hour_of_day, is_weekend, stop_index, historical_load, weather_factor].
- **Database Storage:** Schema is prepared at /forecast_records/{corridorId}/{date} via saveCorridorForecast().

7. Database Security Rules & Indexing

Security rules are declared in database.rules.json:
• system_config, routes: Public read; write restricted to administrators (role === 'admin').
• users/$uid: Read and write restricted strictly to the authenticated user owning the record or admin.
• jeepneys/$jeepney_id: Public read; write restricted to authorized operators and administrators.
• live_telemetry: Public read; write open to edge hardware. Indexed on ['routeId', 'capacityPct', 'lastUpdate', 'isOnline'].
• waiting_commuters: Public read; indexed on ['routeId', 'status', 'expiresAt'].

8. Connected Hardware & Data Authenticity Audit

• **Zero Sample/Mock Vehicles:** All dummy vehicles (including JEEP-VAL-1102) and legacy nodes (/vehicles) were permanently purged from Firebase. Automatic mock data seeding is disabled.
• **Connected ESP32 Hardware:** The database holds exactly one real, physical hardware unit and jeepney:
Device ID: DEV-ESP32-01 | **IP:** 192.168.100.173:81/stream | **Status:** STREAMING
Associated Jeepney: JEEP-NCR-0482 | **Plate:** NDL-8421 | **Seats:** 22 | **Coop:** MAVANODA

9. Primary Source File Directory for Incoming Agents

Component	Source File Path
Firebase Config	src/services/firebase.ts
Database Service Layer	src/services/databaseService.ts
TypeScript Schemas	src/types/database.ts
Security Rules	database.rules.json
Auth Service	src/services/authService.ts
Live Map UI	src/components/MainInterface.tsx
Wait Broadcasting	src/components/waiting/WaitingScreen.tsx
Demand Heatmap	src/components/operator/OperatorDemandMapView.tsx
Hybrid Weather	src/services/hybridWeatherService.ts
Camera Vision Backend	camera_vision_backend/main.py
Camera Vision Config	camera_vision_backend/config.py
ESP32-CAM Firmware	camera_vision_backend/esp32cam_live/esp32cam_live.ino

10. Compact Database Tree Handoff

```
FLOWJEEP REALTIME DATABASE (flowjeep-bf08a)
■■■ system_config
■   ■■■ activeFareMatrix (baseFareFirst4km: 15, succeedingKmRate: 2.2, discountRate: 0.2)
■   ■■■ gpsStaleThresholdSeconds: 45
■   ■■■ pingExpirationMinutes: 20
■■■ users
■   ■■■ {uid} (email, name, role: 'commuter'|'operator', cooperative, assignedRouteId, assignedPlate)
■■■ routes
■   ■■■ {routeId} (routeCode, name, origin, destination, baseFare: 15, stops: map)
■■■ jeepneys
■   ■■■ JEEP-NCR-0482 (plateNumber: NDL-8421, deviceId: DEV-ESP32-01, model, totalSeats: 22, status)
■■■ devices
■   ■■■ DEV-ESP32-01 (cameraStatus: STREAMING, esp32CamIp: 192.168.100.173, firmwareVersion: 1.4.2)
■■■ live_telemetry
■   ■■■ {jeepneyId} (currentCoord: [lat, lng], speedKmh, headingDegrees, occupiedSeats, capacityPct)
■■■ trips [Schema ready / in service]
■   ■■■ {tripId} (jeepneyId, routeId, startTime, endTime, totalBoarded, totalAlighted, peakOccupancy)
■■■ doorway_events
■   ■■■ {eventId} (jeepneyId, eventType, trackingId, confidence, currentOccupancy, timestamp)
■■■ waiting_commuters
■   ■■■ {pingId} (userId, routeId, pickupCoords, destinationCoords, status: 'waiting', expiresAt)
■■■ demand_hotspots
■   ■■■ {corridorId}/{hotspotId} (stopName, lat, lng, waitingCount, demandLevel)
■■■ forecast_records
    ■■■ {corridorId}/{dateKey} (dateString, weatherCondition, peakPercentage, periods)
```